

FleetSec — Documento Maestro de la Solución

Prueba Técnica · Ingeniero Senior de Ciberseguridad · Simon Movilidad / Quantum Data Processing

Autor: Ingeniero de Ciberseguridad (candidato) · **Fecha:** 2026-08-23 **Repositorio:** entregado como ZIP (ver `scripts/init-git-history.sh` para el historial atómico) · **Video:** guion en `docs/video-script.md`

Resumen ejecutivo (para el evaluador)

FleetSec opera telemetría de +60.000 vehículos con datos personales bajo Ley 1581 e ISO 27001 en proceso, **sin programa de seguridad maduro** y con un **breach activo**. La solución aborda los cinco frentes de la prueba con artefactos **ejecutables y verificados**, no diapositivas:

- **Pipeline DevSecOps** funcional en GitHub Actions, con todos los stages (SAST, SCA, SBOM, DAST autenticado, escaneo de imagen, IaC, secretos), quality gates, break-glass auditado y **diseño ≤15 min** por paralelización + caché.
- **VAPT** sobre app propia con las **10 clases CWE**, **10/10 remediadas en código propio**, cada fix con prueba doble (payload malicioso → rechazo, flujo legítimo → OK). **20/20 tests en verde**. Informe en PDF.
- **Hardening AWS como IaC** (módulo Terraform `security-baseline`): **Checkov 254 passed / 0 failed / 12 skips documentados**, tabla de cumplimiento CIS/ISO/Ley 1581.
- **Detección e IR** sobre la línea de tiempo: IOCs enriquecidos, Threat Intel Set de GuardDuty, **4 reglas Sigma validadas**, playbook NIST 800-61 con CLI exacto, **8 técnicas MITRE ATT&CK**, RCA, resumen CEO y plan P1/P2/P3.
- **Documentación:** README, 5 ADRs, diagramas as-is/to-be y C4, Reporte de IA honesto, y este documento maestro.

Filosofía rectora: *criterio de seguridad por encima del cumplimiento literal*. La prueba contiene trampas deliberadas; abajo se detallan y se corrigen.

Evidencia de verificación (ejecutada en este entorno):

Comprobación	Resultado
<code>node --test</code> (PoC + remediación)	20 / 20 pass
Semgrep reglas propias sobre <code>/vuln</code>	dispara (SQLi taint, SSRF taint); 0 en <code>/secure</code>
Semgrep gate producción (ERROR)	0 hallazgos
Checkov Terraform	254 passed / 0 failed / 12 skips documentados
Sigma (<code>pysigma</code>)	4 / 4 reglas válidas
HCL parse (12 archivos) + YAML (workflows, compose, sigma, openapi)	OK
Historial git atómico (script)	15 commits, working tree limpio

1. Análisis crítico y "trampas" de la prueba

Un ingeniero senior detecta requerimientos inseguros y los corrige. Las diez trampas identificadas y su mitigación profesional (detalle en [docs/adr/0005-handling-test-traps.md](#)):

1. **KMS "key policy sin Principal AWS *"**. Un `Principal: "*"` en la política de una llave la vuelve utilizable por cualquiera. **Corrección:** statement de administración al `account root` + principals de servicio scopeados (`logs.<region>.amazonaws.com`), nunca comodín. (`terraform/modules/security-baseline/kms.tf`)
2. **"sin SG con 0.0.0.0/0 excepto 80/443 en ALB"**. **Corrección:** solo el SG del ALB expone 0.0.0.0/0, exclusivamente en 80 (redirect → 443) y 443; los tiers app y data solo aceptan del SG anterior; puertos admin (22/3389) jamás. Enforced por Checkov CKV_AWS_24/25 (probado: pasan mis SG, falla un SG malo).
3. **RDS con `ssl=1` y `log_connections=1`**. Config aparentemente pedida, fácil de omitir. **Corrección:** parameter group con `rds.force_ssl=1` , `log_connections=1` , `log_disconnections=1` , + IAM auth, + sin endpoint público, + backups 7d. (`rds.tf`)
4. **V-10: "migrar a variable de entorno o gestor de secretos — nunca mover a otro archivo del repo"**. Mover el secreto a un `.env` / `.js` versionado sigue siendo exposición. **Corrección:** el secreto se **elimina del código** y se lee de env/AWS Secrets Manager (rotación 30d). (`app/src/routes/secure.js` , `rds.tf`)
5. **svc-monitoring (cuenta de servicio) con `ConsoleLoginSuccess`** en la línea de tiempo. Las cuentas de servicio **no** deben tener consola. **Corrección:** el playbook revoca el login profile y el backlog P1 añade un SCP que lo prohíbe.
6. **DAST "cobertura ≥80% del OpenAPI"** sin OpenAPI provisto. **Corrección:** se publica `app/openapi.yaml` de la superficie *segura* para el escaneo autenticado con Bearer JWT.
7. **Break-glass "2 aprobadores + registro auditado"**. Un bypass sin auditoría es una puerta trasera. **Corrección:** Environment protegido (2 reviewers de CODEOWNERS) + issue CRITICAL automático por cada override. (`.github/workflows/break-glass.yml`)
8. **Reporte de IA: "afirmar que no hubo errores no suma puntos"**. **Corrección:** se **admite** una alucinación real (digest Docker inventado) y otros errores; §7 y `docs/ai-report.md` .
9. **App vulnerable pedida (SQLi sin parametrizar, `alg:none` , etc.)**. Construir y dejarla expuesta sería el error. **Corrección:** el laboratorio `/vuln` está *gated* (`LAB_MODE`), **nunca** se despliega (`LAB_MODE=false` en el Dockerfile) y convive con su gemelo remediado `/secure` .
10. **"Pipeline ≤15 min" con escaneos pesados**. Un pipeline serial no cumple. **Corrección:** fan-out en paralelo, caché npm/Trivy/Semgrep, imagen slim sin build nativo (`sql.js` WASM), filtros de ruta.

2. Desarrollo técnico por entregable

Entregable 01 — Pipeline DevSecOps (`.github/workflows/devsecops.yml`)

- **SAST:** Semgrep con **4 reglas propias** en *taint mode* (source `req.*` → sink `db.exec / fetch`), más `p/owasp-top-ten` . Gate bloqueante sobre `app/src` (0 CRITICAL/HIGH) con supresiones documentadas (`razón·fecha·responsable`); run informativo full-repo que evidencia las reglas.
- **SCA + SBOM:** Trivy fs (bloquea HIGH/CRITICAL) + **CycloneDX** en cada build.
- **Secretos:** Gitleaks con allowlist documentado para el artefacto de laboratorio V-10.

- **Contenedor:** build multi-stage, base **pinneada**, non-root, **read_only** , **cap_drop ALL** ; Trivy imagen bloquea CRITICAL.
- **DAST autenticado:** OWASP ZAP API scan con Bearer JWT sobre el OpenAPI; HIGH/CRITICAL bloquean, MEDIUM abre issue **security/medium** .
- **laC:** **terraform validate** + Checkov (hard-gate; bloquea SG 0.0.0.0/0 en 22/3389).
- **Break-glass:** 2 aprobadores + issue de auditoría.
- **≤15 min:** jobs en paralelo → **security-gate** (fan-in), caché y **cancel-in-progress** .

Entregable 02 — VAPT y remediación (**app/ , **vapt/vapt-report.pdf**)**

App Express (Node 20) con **/vuln** (10 CWE) y **/secure** (remediado). Cada hallazgo tiene ficha con **vector CVSS 3.1 completo**, OWASP 2021, CWE, PoC funcional, impacto C/I/D + Ley 1581, y fix con test doble. Los CVSS fueron **recalculados** con calculadora propia.

ID	Vuln	CVSS	Fix
V-01	SQLi (CWE-89)	9.8	prepared statements + validación entero
V-02	JWT alg:none (CWE-345)	9.1	allowlist de algoritmos
V-03	SSRF (CWE-918)	8.8	allowlist host/esquema + bloqueo IP privada + IMDSv2
V-04	XXE (CWE-611)	8.6	entidades externas deshabilitadas
V-05	Mass Assignment (CWE-915)	8.1	DTO allowlist (stripUnknown)
V-06	Path Traversal (CWE-22)	7.5	canonicalización + contención
V-07	Rate Limiting (CWE-307)	7.5	rate limiter + WAF
V-08	PII en logs (CWE-359)	7.5	sanitizador PII en todos los niveles
V-09	IDOR (CWE-639)	6.5	verificación de propiedad
V-10	Hardcoded creds (CWE-798)	9.1	env / Secrets Manager

10/10 remediadas · 20/20 tests. Informe completo: **vapt/vapt-report.md** (+ PDF).

Entregable 03 — Hardening AWS IaC (**terraform/)**

Módulo **security-baseline** : IAM mínimo (sin AdministratorAccess/wildcards; password policy 14/90/24), S3 (BPA + SSE-KMS CMK + Object Lock COMPLIANCE para logs + TLS-only + lifecycle Glacier), VPC 3 capas (data aislada sin NAT, NACLs, Flow Logs, VPC endpoints), RDS (Multi-AZ, CMK, sin público, IAM auth, PI + enhanced monitoring), KMS (CMK por servicio, rotación anual), Secrets Manager (rotación 30d), CloudTrail (multi-region + log file validation + Object Lock + métricas/alarmas root/IAM/SG/KMS/DeleteTrail), AWS Config (recorder + 7 managed rules), GuardDuty (all-region + S3 + malware + Threat Intel Set), Security Hub (FSBP + CIS 1.4), WAFv2 (SQLi/BadInputs BLOCK, Common en COUNT justificado, rate limit 1000/5min, geo CO/PE/US). **Checkov: 254/0/12**. Tabla de cumplimiento: **docs/compliance-matrix.md** (27 controles PASS).

Entregable 04 — Detección e IR (**detection/)**

IOCs enriquecidos (VirusTotal/AbuseIPDB/Shodan/Tor list) y carga a **GuardDuty Threat Intel Set** vía Terraform + CLI. **4 reglas Sigma** validadas (IAM admin off-hours, DeleteTrail CRITICAL, SQLi en app logs, bulk S3 GetObject). Playbook NIST 800-61 con **CLI exacto y reversible**

(preservar → contener → erradicar → recuperar → lecciones), **8 técnicas MITRE ATT&CK v14**, RCA 5-Whys + Swiss-cheese, resumen CEO ≤1 pág. y notificación SIC (Ley 1581, 15 días hábiles). Plan P1/P2/P3.

Entregable 05 — Documentación

README completo, 5 ADRs, diagramas (as-is/to-be, C4 objetivo con trust boundaries, secuencia del breach, flujo del pipeline), Reporte de IA, y este documento maestro.

3. Plan de sprints (resumen)

Sprint	Foco	DoD clave
0	Fundaciones, ISMS, contención inicial	Repo + hooks; secretos rotados; commit atómico
1	Pipeline DevSecOps (E01)	Todos los stages verdes; SBOM; <15 min; break-glass
2	VAPT y remediación (E02)	≥8/10 remediados (logrado 10/10); PDF; 20/20 tests
3	Hardening AWS IaC (E03)	HCL válido; Checkov 0 failed; tabla ≥10 controles
4	Detección e IR (E04)	4 Sigma válidas; playbook CLI; ≥6 ATT&CK; SIC
5	Docs + sustentación (E05)	README + ADRs + diagrama + video + Reporte IA

Detalle completo (objetivos, alcance, entregables, controles): [docs/sprints.md](#).

4. Guion de video (≤10 min, cámara activa)

Guion palabra por palabra, con marcas de tiempo y qué mostrar en pantalla: [docs/video-script.md](#).
Cubre: introducción, arquitectura general, trampas, pipeline en ejecución, walkthrough de 2 vulnerabilidades (SQLi y JWT) explotadas y parcheadas, y respuesta al breach de la línea de tiempo. Instrucciones de grabación y publicación (YouTube No listado): [docs/instrucciones-candidato.md](#).

5. Instrucciones de ejecución para el candidato

Lo que la IA no puede hacer por ti (crear el repo remoto, grabar el video, generar el PNG de draw.io, aplicar Terraform en tu nube) está paso a paso en [docs/instrucciones-candidato.md](#). Resumen: 1. `bash scripts/init-git-history.sh` → historial atómico Conventional Commits. 2. `gh repo create ... --push` → publicar el repo. 3. Generar `architecture.png` desde el Mermaid en draw.io (pasos incluidos). 4. Grabar con OBS siguiendo el guion; subir a YouTube **No listado**; pegar el link en el README.

6. Reporte de IA (obligatorio, resumen)

Se usó IA (Claude) para generar y acelerar; **cada artefacto se verificó con herramientas reales**.
Alucinación admitida: el Dockerfile inicial fijó un **digest SHA256 inventado** que habría roto `docker build`; se detectó (un digest solo sale de `docker inspect`) y se corrigió a un tag pinneado real. Otros errores corregidos: reglas Semgrep que no disparaban (→ taint), modificador Sigma inválido `nocase`, y un check Checkov en Python que no cargaba (→ enforcement por CKV_AWS_24/25 + política YAML + self-test). **No delegaría a IA sin supervisión:** IR en vivo, aceptación de riesgo/decisiones de cumplimiento,

manejo de secretos reales, aprobación de excepciones, y exploits contra terceros. Detalle: `docs/ai-report.md`.

7. Desafíos y próximos pasos

Desafíos: convivencia de laboratorio vulnerable con gates de 0 HIGH/CRITICAL (resuelto con supresiones documentadas + separación `/vuln` | `/secure`); XXE portable sin libs nativas (parser de laboratorio declarado); custom check Checkov Python que no cargaba (enforcement por checks nativos + YAML + self-test); `terraform plan` completo requiere credenciales (HCL + Checkov validados offline).

Próximos pasos: federación OIDC (eliminar llaves estáticas), DNS Firewall + egress firewall, SOC/SIEM 24x7 con las Sigma desplegadas, y camino a certificación ISO 27001 (SoA, ISO 27005, auditoría interna ISO 19011), cross-region replication del bucket de telemetría.